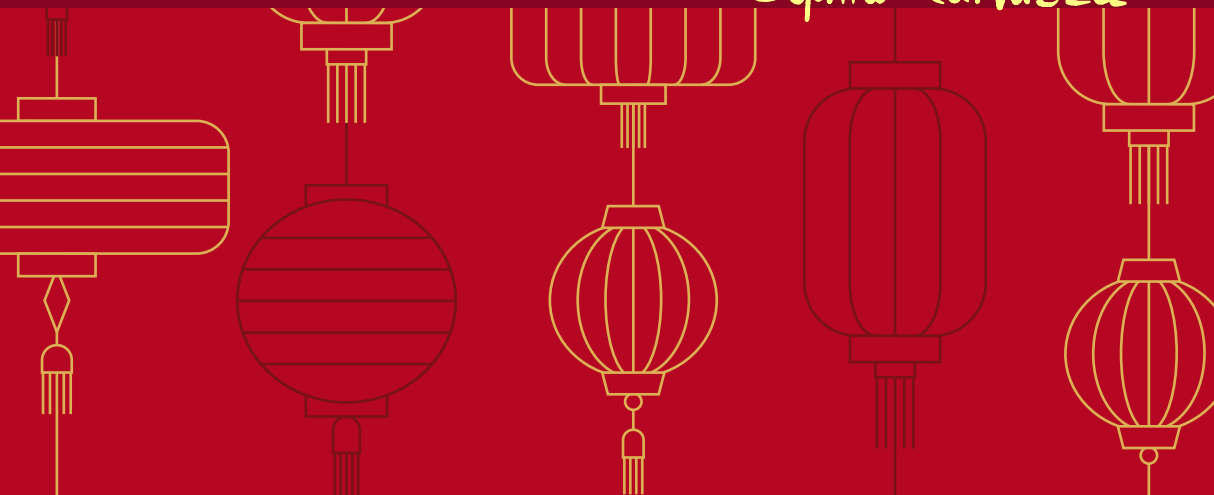
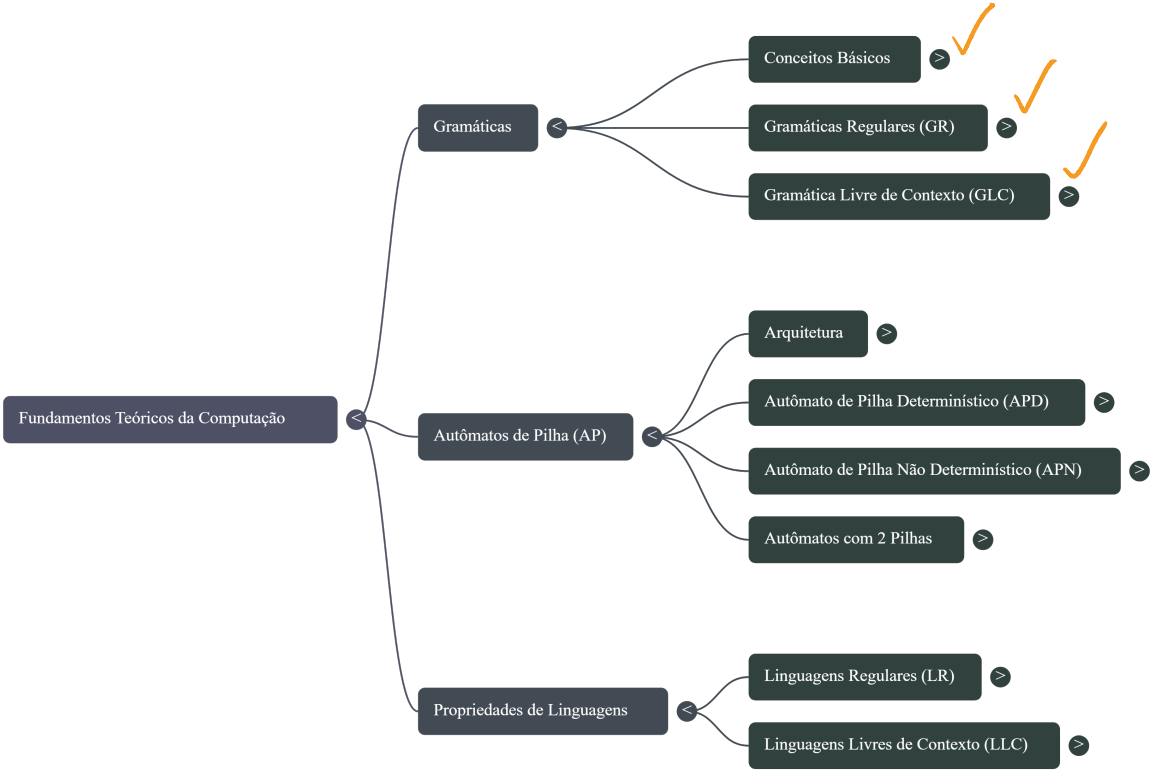
Riassunto Prova II FTC

Sophia Carvazza



Matéria :

- gramática
- gramática e linguagem regular
- gramática e linguagem livre de contexto
- derivação (árvore de parser)
- ambiguidade de gramática (+ de 1 árvore de parser diferente pra mesma string)
- gerar gramática a partir de uma linguagem
- máquina/ autômato de pilha
- forma normal de chomsky
- lema do bombeamento



- Fundamentos Teóricos da Computação
  - Gramáticas
    - Conceitos Básicos
      - Símbolos Não-Terminais (Variáveis): Ex: S, A, B
      - Símbolos Terminais (Alfabeto): Ex: a, b, c, λ
      - Formas Sentenciais: Símbolos Terminais e Não-Terminais
      - Sentenças: Apenas Símbolos Terminais
      - Produção (Regra): Par ordenado [B, bA] ou B → bA
      - Derivação
      - Linguagem Gerada (L(G))
    - Gramáticas Regulares (GR)
      - Equivale a Gramática Linear à Direita (GLD) ou à Esquerda (GLE)
      - Gramática Linear à Direita (GLD)
      - Gramática Linear à Esquerda (GLE)
    - Gramática Livre de Contexto (GLC)
      - Definição: G = (V, Σ, P, S), Regras: X → w (X ∈ V, w ∈ (V ∪ Σ)*)
      - Linguagem Livre de Contexto (LLC): L = L(G)
      - Exemplos de LLC
      - Derivações e Ambiguidade
      - Forma Normal de Chomsky (FN Chomsky)
  - Autômatos de Pilha (AP)
    - Arquitetura
      - Componentes Comuns (AF e AP): Fita de Leitura Unidirecional
      - Componentes Comuns (AF e AP): Controle + Função de Transição
      - Componentes Comuns (AF e AP): Registrador de Estado Atual
      - Componentes Específicos do AP: Pilha
    - Autômato de Pilha Determinístico (APD)
      - Definição: Sêxtupla M = (E, Σ, Γ, δ, i, F)
      - Restrição: δ não deve possuir transições compatíveis
      - Simulação (Algoritmo): Usa 'MarcaFundo' na pilha, aceita se Símbolo=EOS e topo='MarcaFundo' e Estado ∈ F
      - Configuração Instantânea
      - Linguagem Reconhecida (L(M)): Por Pilha Vazia e Estado Final
    - Autômato de Pilha Não Determinístico (APN)
      - Definição: Sêxtupla M = (E, Σ, Γ, δ, i, F)
      - Função de Transição: δ → P(E × Γ*)
      - Propriedade: δ pode possuir transições compatíveis
      - Linguagens Reconhecidas (APN)
    - Autômatos com 2 Pilhas
      - Verifica o topo de cada pilha em cada transição
      - Reconhecem linguagens mais complexas (Ex: {a^i b^i c^i | i ≥ 0})
  - Propriedades de Linguagens
    - Linguagens Regulares (LR)
      - Fechamento
      - Aplicações de Fechamento
      - Lema do Bombeamento (LRs)
    - Linguagens Livres de Contexto (LLC)
      - Fechamento
      - Pumping Lemma (LLC)

Gramática → usada p/ se especificar (gerar) uma linguagem

★ ex: gramática regular → linguagem regular

$S \rightarrow Aa$

- S: símbolos não terminais (auxiliares da geração de sentenças)
- $\rightarrow$: produção / regra
- Aa: símbolos terminais (formadores das sentenças → alfabeto)

$G = A \rightarrow bCa$

- bCa: formas sentenciais (palavras formadas por símb. terminais e não-terminais)

$C \rightarrow ccc$

- ccc: sentenças (palavras formadas apenas por símb. terminais)

★ **derivação** → obtenção de uma forma sentencial (bca) a partir de outra (ex: dado $B \rightarrow bA$ e cBc, derivemos: $cBc \rightarrow cbAc$)

> $S \Rightarrow w$ significa que w é derivado a partir da variável S em um passo
>
> $S \overset{*}{\Rightarrow} w$ significa que w é derivado a partir da variável S em zero ou mais passos

★ composição da gramática → $G = (V, \Sigma, P, S)$

- V = símbolos não terminais (variáveis)
- Σ = símbolos terminais (alfabeto, $V \cap \Sigma = \emptyset$)
- P = produções / regras
- S = símbolo inicial (variável de partida)

★ linguagem gerada por uma gramática → $L(G) = \{w \in \Sigma^* \mid S \overset{*}{\Rightarrow} w\}$

Recursão

Regras recursivas permitem que uma gramática gere infinitas palavras.

Recursão { Direta : $A \rightarrow aA$; Indireta : $A \overset{*}{\Rightarrow} w \overset{*}{\Rightarrow} aA$ (e w não contém A) }

★ (Seja $G = (\{S, A\}, \{a, b\}, \{S \rightarrow AA, A \rightarrow AAA \mid bA \mid Ab \mid a\}, S)$)

Derivação Mais a Direita (DMD)

$\Rightarrow S \rightarrow AA \rightarrow Aa \rightarrow AAAa \rightarrow AAbAa \rightarrow AAbaa \rightarrow AbAbaa \rightarrow Ababaa \rightarrow ababaa$

Durante o processo de derivação o não-terminal expandido de cada forma sentencial é sempre o mais a direita.

Derivação Mais a Esquerda (DME)

$\Rightarrow S \rightarrow AA \rightarrow aA \rightarrow aAAA \rightarrow abAAA \rightarrow abaAA \rightarrow abAbAA \rightarrow ababAA \rightarrow ababaA \rightarrow ababaa$

Durante o processo de derivação o não-terminal expandido de cada forma sentencial é sempre o mais a esquerda.

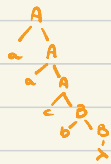
REGULARES

GLD (gramática linear à direita) vs GLE (gramática linear à esquerda)

GLD: $G = (V, \Sigma, P, S)$, em que a forma:

$A \to a$
$A \to aB$
$A \to \lambda$

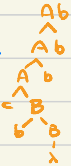
↳ ex: p/ gerar $L = a^*cb^*$
$A \to aA \mid cB$
$B \to bB \mid \lambda$



GLE: $G = (V, \Sigma, P, S)$, em que a forma:

$A \to a$
$A \to Ba$
$A \to \lambda$

↳ ex: p/ gerar $L = a^*cb^*$
$A \to Ab \mid c$
$B \to Ba \mid \lambda$



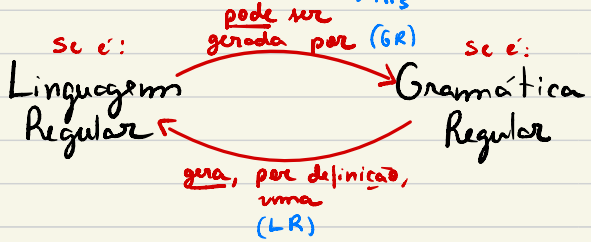
→ Nível 3 da Hierarquia de Chomsky

Gramática Regular → uma gramática é regular se é uma GLD ou GLE (se está em um destes formatos). Ela proíbe produção com recursão bilateral (ex: $P \to 0P1$) ou múltiplas variáveis no lado direito (ex: $A \to BC$);

↳ a principal característica que define uma GR é essa restrição estrutural (só é GR se for uma GLD ou uma GLE)

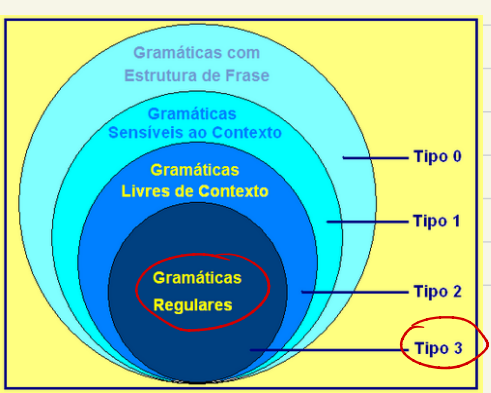
↳ a gramática regular é uma forma de especificar as linguagens regulares.

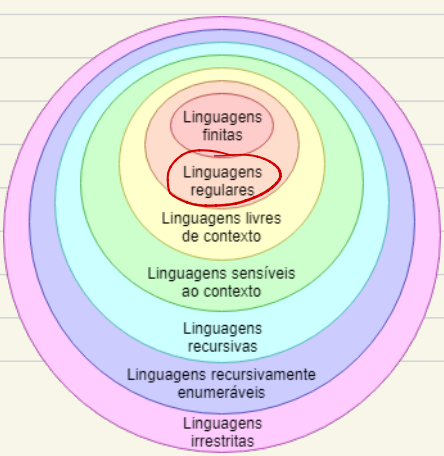
↳ é a classe menos poderosa na hierarquia de Chomsky, pq tem a restrição linear que garante a geração de linguagem regular (reconhecidas por Autômatos Finitos que têm memória limitada) ↳ AF_s



*ex: se a gramática precisar gerar uma contagem infinita ou empilhar/desempilhar sinetricamente, ela precisa de uma produção + complexa ⇓

GLC = Gramática Livre de Contexto





Ex. de linguagens não regulares: linguagens que exigem balanceamento* ou comparação de contagem, como: (* e portanto não podem ser gerados por uma GR.)

- $L_1 = \{a^n b^n \mid n \geq 0\}$
- $L_2 = \{a^m b^n \mid m \leq n\}$, $L_3 = \{a^m b^n \mid m \geq n\}$, $L_4 = \{a^m b^n \mid m \neq n\}$
- $L_5 = \{ww \mid w \in \{a, b\}^*\}$
- $L_6 = \{w \in \{a, b\}^* \mid n_a(w) = n_b(w)\}$ ⇒ (nº de a's = nº de b's)
- $L_7 = \{a^K b^m c^n \mid K = m + n\}$ ⇒ soma de contagens

Características de linguagens não-regulares:

1- contagem, balanceamento ou comparação (igualdade ou relação de ordem) = o AF autômato precisa conter (um nº arbitrário (variável) ilimitado) e lembrar a qntd de um símbolo p/ compará-lo a outro (=, >, <, etc) e garantir alguma condição.

↳ ex: • $L = \{a^n b^n\}$ ⇒ (igualdade - precisa lembrar o valor de n)
• $L = \{a^m b^n \mid m \leq n\}$ ⇒ (ordem - precisa contar m e n)

2- linguagens de estruturas de padrão simétrico (repetição de palavras) = qualquer palavra que seja repetição de si mesma, pq ele não é capaz de lembrar a primeira metade (que pode ter tam. ilimitado) p/ comparar c/ a segunda.

↳ ex: • $L = \{ww \mid w \in \{a, b\}^*\}$

3- linguagens baseados em propriedades numéricos complexos (unário) = se a propriedade do expoente é complexa e exige rastreamento infinito. O AF não consegue distinguir entre o nº de símbolos grande o suficiente p/ ser um quadrado ou primo, por ex.

↳ ex: • $L = \{a^{n^2} \mid n \geq 0\}$ ⇒ quadrados perfeitos
• $L = \{a^n \mid n$ é primo$\}$ ⇒ nºs primos

✱ Linguagens Regulares que parecem que não são:

1- **linguagens com restrição posicional fixa** (olhar p/ trás) = Quando o limite de "olhar p/ trás" for fixo, um nº finito de estados é suficiente.
↳ ex: $L = \{w \in \{0,1\}^* \mid 3^{\circ}$ dígito de w, da direita p/ esquerda, é $1\}$

2- **linguagens baseadas em módulo** = Quando a restrição (numérica) é verificada por um resto de divisão, é regular.
↳ ex: $L = \{w \in \{0,1\}^* \mid w$ representa um número binário divisível por $6\}$

3- **linguagens que usam contagem de forma não-comparativa** = Quando possui um nº fixo p/ contagem ou ilimitado sem dependência interna
↳ ex: $L = \{a^m b^n \mid m$ é par$\} \Rightarrow (a^{2k}b^*)$

Característica	Implica	Exemplo (Baseado em Fontes)
Memória Finita Suficiente (LR)	Regras fixas, propriedades modulares, padrões locais.	L_2: Número binário divisível por 6 [3].
Memória Infinita Necessária (LNR)	Relações de igualdade ou ordem entre contagens de símbolos, ou padrões globais de tamanho ilimitado.	$\{a^n b^n \mid n \geq 0\}$ [4]; $\{a^m b^n \mid m \neq n\}$ [9]; $\{ww \mid w \in \{a,b\}^*\}$ [11].

✱ Lema do Bombeamento p/ LRs → no resumo anterior

Gramática Livre de Contexto → permite recursão irrestrita no lado direito da produção, desde que haja apenas um símbolo não-terminal no lado esquerdo.

↳ ex: $P \rightarrow OPL$

↳ isso permite a geração de estruturas aninhadas ou balanceadas

★ A aplicação da regra em X não depende do que está ao redor de X, apenas do próprio X (livre de contexto)

★ Uma GLC é uma quádrupla $G = (V, \Sigma, P, S)$ (símbolos não-terminais; terminais; produção; símbolo inicial)

↳ Toda regra $r \in P$ é da forma $X \rightarrow w$, onde X é um único símbolo não-terminal ($X \in V$) e w é uma string de qualquer combinação de variáveis e terminais (incluindo λ)

$A \rightarrow \lambda$
$A \rightarrow aA$
$A \rightarrow AA$
$A \rightarrow a$

★ GLCs (gramáticas livres de contexto) — geram → LLCs (linguagens livres de contexto) — reconhecidas por → Autômatos de Pilha

GLCs (gramáticas livres de contexto) — geram → LRs (linguagens regulares) — reconhecidas por → Autômatos Finitos

> Linguagem Livre de Contexto (LLCs) → tratam questões típicas de linguagens de programação, e contém a classe dos LRs.

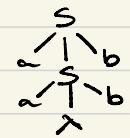
↳ Pode ser representada por um Autômato de Pilha ou por uma Gramática L.C. (GLC)

✱ Árvores de Derivação (de uma forma sentencial G): é uma árvore ordenada construída recursivamente assim:

1. uma árvore sem arestas cujo único vértice tem rótulo S é uma AD de S;
2. se $X \in V$ é rótulo de uma folha f de uma AD A, então:
   - se $X \to \lambda \in P$, logo a árvore obtida acrescentando-se a A mais um vértice v com rótulo λ e uma aresta $\{f, v\}$ é uma AD;
   - se $X \to x_1 x_2 \dots x_n \in P$, em que $x_1, x_2, \dots, x_n \in V \cup \Sigma$, logo a árvore obtida acrescentando-se a A mais n vértices $v_1, v_2, \dots, v_n$ com rótulos $x_1, x_2, \dots, x_n$, nesta ordem, e n arestas $\{f, v_1\}, \{f, v_2\}, \dots, \{f, v_n\}$ é uma AD.

✱ se a sequência dos rótulos da fronteira de AD é igual a forma sentencial w, AD é uma árvore de derivação de w.

↳ ex: aabb



Ambiguidade de Gramática

↳ Uma GLC é dita ambígua quando existe + de 1 Árvore de Derivação (AD) p/ alguma palavra/sentença gerada por G.

(ou seja, se tem várias formas de atribuir uma estrutura a uma mesma palavra)

≠ ADs → ≠ Estruturas Sintáticas → ≠ Significados p/ a mesma string

✱ Se a gramática é ambígua, o analisador sintático não consegue determinar qual operação deve ser executada primeiro, resultando em estruturas semânticas conflitantes.

Verdadeiro Fator de decisão! →

✱ Uma GLC é ambígua se, e somente se:

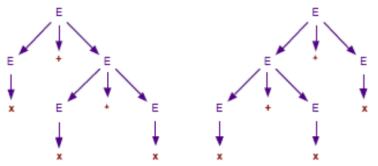
1 - Existe + de 1 AD (árvore de derivação)
2 - Existe + de 1 DMD (derivação + à direita)
3 - Existe + de 1 DME (derivação + à esquerda)

} pq essas duas são uma forma de AD

↳ Forma + comum de provar ambiguidade ⇒ encontrar 2 ADs p/ mesma string!

Existe uma única DMD e DME (iguais!) correspondente a uma AD. Se conseguirmos construir duas ADs c/ estruturas ≠, as DMs podem ou não continuar sendo as mesmas.

♦ *Exemplo*: x+x*x



- E ⇒ E+E ⇒ X+E ⇒ x+E*E ⇒ x+x*E ⇒ x+x*x
- E ⇒ E+E ⇒ E+E*E ⇒ E+E*x ⇒ E+x*x ⇒ x+x*x
- E ⇒ E+E ⇒ E+E*E ⇒ X+E*E ⇒ x+x*E ⇒ x+x*x
- etc...

- derivação mais à esquerda
  - E ⇒ E+E ⇒ x+E ⇒ x+E*E ⇒ x+x*E ⇒ x+x*x
  - E ⇒ E*E ⇒ E+E*E ⇒ x+E*E ⇒ x+x*E ⇒ x+x*x
- derivação mais à direita
  - E ⇒ E+E ⇒ E+E*E ⇒ E+E*x ⇒ E+x*x ⇒ x+x*x
  - E ⇒ E*E ⇒ E*x ⇒ E+E*x ⇒ E+x*x ⇒ x+x*x

nem sempre é! — 2× — 2× — 2× — (Deveria ser só 1 de cada)

Gramática Ambígua → Linguagem Ambígua ⇒ pode ser gerada por pelo menos uma G. ambígua → mas pode possuir gramáticas não-ambíguas

Linguagem Ambígua → Linguagem não-ambígua

uma ambiguidade independe da outra

★ linguagens inerentemente ambíguas: Qualquer gramática que a gere é ambígua. É impossível achar uma GLC que a gere sem ambiguidade.
↳ ex: $L = \{a^n b^m c^m \mid n,m \geq 0\} \cup \{a^n b^n c^m \mid n,m \geq 0\}$

> Autômato de Pilha → reconhece LLCs e possui componentes adicionais, sendo capaz de ter uma memória limitada.

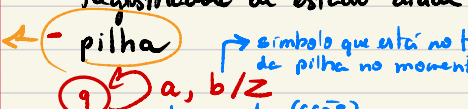
- fita de leitura unidirecional
- controle + função de transição
- registrador de estado atual
- pilha

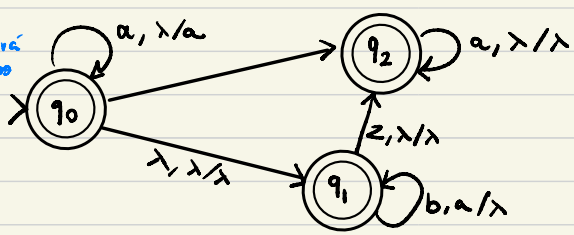
$M = (E, \Sigma, I, \delta, i, F)$ — transição; estados; alfabeto; alfabeto da pilha; estado inicial; conjunto de estados finais

específico do AP

símbolo que está no topo da pilha no momento

ex: $\{a^n b^n a^m \mid n,m > 0\}$





★ se a = λ → transição espontânea, não precisa de input

★ se Z = λ → representa "desempilhar" o símbolo b

★ se b = λ → transição ocorre sem consulta à pilha (nada acontece)

★ se b = λ e Z ≠ λ → representa "empilhar" o símbolo Z.

uma string só é aceita se: a pilha estiver vazia & ela estiver no estado de aceitação!

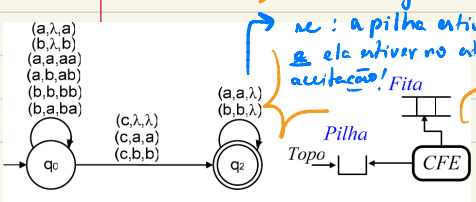
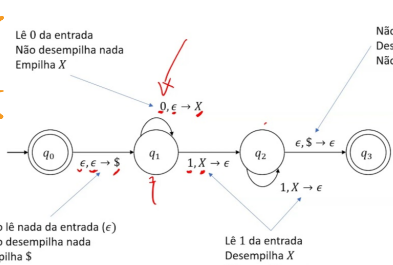


Diagrama de estados — $0^n 1^n$



Autômato de Pilha Não-Determinístico (APN) VS Autômato de Pilha Determinístico (APD)

- + poderoso → reconhece todos as LLCs e geralmente é o considerado AP "normal"
- A função de transição mapeia um conjunto de possíveis próx. estados e cadeias a empilhar.
- Ele pode ter transições compatíveis ou não. Se houver compatibilidade, ele "ramifica" a computação p/ seguir todos cam. possíveis.

$L = \{0^m 1^n \mid m \geq n\}$

$L = \{w \in \{0,1\}^* \mid w = w^R\}$

Toda L reconhecida por um APN → LLC, e vice-versa!

- − poderoso e + restrito, mas é mais simples de implementar, pq aplica o determinismo.*
  *sempre há apenas 1 transição válida
- A função de transição é definida p/ mapear para um único resultado.
- O APD não pode possuir transições compatíveis.
- "Marca Fundo" p/ ident. o fundo da pilha

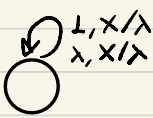
$L = \{a^n b^n \mid n \geq 0\}$

$L = \{0^m 1^n \mid m \leq n\}$

Toda L reconhecida por um APD → LLC

MAS

Nem toda LLC é reconhecida por um APD



★ Transições Compatíveis → Quando, em um mesmo estado, múltiplas regras se aplicam à mesma entrada ou topo de pilha (incluindo o uso de λ). → elas criam ambiguidade sobre qual caminho o AP deve seguir.

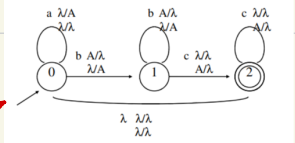
p/ 2 trans. serem compatíveis, devem cumprir:

deve cumprir ambas!
- condição de entrada é compatível (a=c ou a=λ ou c=λ)
- condição de pilha é compatível (b=d ou b=λ ou d=λ)

p/: e, a/b e, c/d

Autômatos de 2 pilhas:

→ Em cada transição o autômato verifica o símbolo no topo de cada pilha e executa uma op. em cada.



★ Nem toda L reconhecida por um AP de 2 pilhas é LC.

Exemplo: autômato para L = { $a^i b^j c^i \mid i \geq 0$ }.

Fechamento de Operações p/ LRs

↳ seja uma classe de linguagens L e uma operação sobre linguagens O. Diz-se que **L é fechada sob O** se a aplicação de O a linguagens pertencentes a L sempre resulta em uma linguagem pertencente a L.

analogamente

Considere duas linguagens regulares L_1 e L_2, então:

- $L_1 \cup L_2$ também é regular (+) "ou"
- $L_1 \cap L_2$ também é regular (×) "e"

- $L_1 L_2$ também é regular (concatenação)
- L_1^* também é regular (repetição)
- $\overline{L_1}$ também é regular (complemento)

Fechamento de Operações p/ LLCs

↳ se L_1 e L_2 são LLCs, o resultado das operações abaixo também será uma LLC.

- Teorema: se L_1 e L_2 são linguagens livres do contexto, então:
  - $L_1 \cup L_2$, $(L_1)^*$, $L_1 L_2$ são LLC, mas (união) (fecho de Kleene) (concatenação)

LLCs são fechadas sob estas operações básicas =
- união
- fecho de Kleene
- concatenação

- Teorema: o conjunto das LLC's não é fechado sob interseção ou complemento:
  1. $L_1 \cap L_2$ pode não ser LLC - $a^i b^j c^k \cap a^k b^k c^i = a^k b^k c^k$ i, k > 0. (interseção)
  2. $(L_1)'$ pode não ser LLC. Prova: (complemento)
     - considere L_1 e L_2 LLC. Faça $L = ((L_1)' \cup (L_2)')'$. Note que o resultado contradiz (1). → $\overline{L_1 \cup L_2} = \overline{L_1} \cap \overline{L_2}$ ⇒ pode não ser LLC!
     - $L = \{ ww \mid w \in \{a,b\}^* \}$ não é LLC. Mas L' é LLC.

LLCs não são fechadas sob estas operações
- interseção
- complemento

!!!

- Teorema: seja L_1 uma LR e L_2 uma LLC, então:
  - $L_1 \cap L_2$ é uma LLC. Prova por construção do PDA.

$L_1 \setminus L_2$ ou $L_1 - L_2$ ↳ $\{w \mid w \in L_1 \text{ e } \notin L_2\}$

L_1 (LR) $\cap$ L_2 (LLC) ⇒ L_3 (LLC)
(interseção)

> A diferença de duas LLCs nem sempre são LLC tb!

essencial p/ aplicar o Lema do bombeamento

Forma Normal de Chomsky

↳ é um formato padronizado e restrito p/ GLCs.

↳ ela serve p/ simplificar e padronizar as regras (útil p/ análise algorítmica e provas de propriedades das LLCs)

↳ (pumping lema)

remove produções vazias, derivações duvidosas, etc

> Uma GLC está na FNC se todas as suas produções pertencem a um desses formatos:

$S \to \lambda$
$A \to a$
$A \to BC$

} toda árvore de parse é uma árvore binária

> Passo a Passo p/ transformar uma G em FNC:

1- Eliminar a recursão do símbolo inicial (garante que S não seja recursivo → adiciona o novo inicial $S' \to S$ ou $S' \to S|\lambda$ (se S for anulável))

evita violação por remoção de λ

2- Eliminar as transições lambda (remove produções da forma $A \to \lambda$ (exceto $S' \to \lambda$, se existir))

gera gramáticas essencialmente não-contráteis (λ tratados)

↳ 1- identificar todos as variáveis que geram λ.
↳ 2- marcar as variáveis que podem gerar λ indiretamente (via outras variáveis marcadas)
↳ 3- criar as novas produções sem as produções λ (criando todas as combinações possíveis sem essas variáveis)
↳ 4- eliminar o λ original.

3- Eliminar as regras de cadeias (remove todas as regras de produção da forma $A \to B$): (A pode derivar uma string de indiretamente)

produções unitárias!

(eliminar 1º as que não dependem de outras regras de cadeia)

↳ seja $A \to B$ uma regra de cadeia
↳ $\forall B \to u$, deve ser add. a regra $A \to u$ ao conj. de produções

Exemplo:

$$G: \begin{cases} S \to A \mid b \\ A \to B \mid a \\ B \to S \mid b \end{cases}$$

produções unitárias:
$S \to A$
$A \to B$
$B \to S$

★ passo a passo detalhado de eliminação de regra de cadeia:

1° → construímos um fecho unitário p/ cada variável

Fecho (S) = { S, A, B }
Fecho (A) = { A, B, S }
Fecho (B) = { B, S, A }

↳ Fecho(S) = todas variáveis que S pode derivar via regras de cadeia (indiretamente)

eliminamos 1° os de fecho menor, que dependem de menos variáveis (p/ reduzir recursão e complexidade no processo)

2° → substituímos as produções unitárias

- p/ eliminar S → A, substituímos A por todas as produções de A que não são unitárias (A → a).
- como S tb alcança B, adicionaremos tb as produção não-unitária de B (B → b). ↳ ficando S → a|b|b ⇒ { S → a|b
- fazemos o mesmo p/ A e B ⟹ { A → a|b
  { B → a|b

4- Eliminar os símbolos inúteis (remove símbolos que não são alcançáveis a partir do símbolo inicial ou que não derivam uma string de terminais)

A ORDEM IMPORTA!!! {

↳ 1° removemos os símbolos que não geram terminais

↳ 2° removemos os símbolos não alcançáveis

↳ removemos todas as produções que envolvem as variáveis removidas

1° conj. que geram terminais = { ", ", " } → 3° remove os que não estão aqui!

conj. dos alcançáveis pelo simb. inicial = { ", ", " } ↳ remove o que não estão aqui!

2° indiretamente: Se a variável tem só produção c/ recursão p/ ela mesma e/ou produz outras que só fogem disso (e nunca produz uma que gera terminal) → não gera terminal!

mas se produz uma que gera terminal (sem chamar outras que não geram junto) → produz terminal

Terminais não podem aparecer misturados c/ não-terminais ou tb + de 1 terminal A → a, A → BC ↳ pq as regras na FNC são limitadas a

5- Isolar terminais (p/ cada produção A → w, onde w contém terminais a, e p/ cada terminal a em w):

↳ cria-se uma nova produção $X_a \to a$ (se não existir)

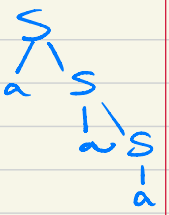
↳ substitui-se o terminal a pelo novo não-terminal X_a em w.

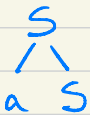
$R_1 : S \to aABC$	$S \to X_aABC$
$R_2 : S \to a$	S → a (FNC)
$R_3 : A \to aA$	$A \to X_aA$
$R_4 : A \to a$	A → a (FNC)
$R_5 : B \to bbB$	$B \to X_bX_bB$
$R_6 : B \to bc$	$B \to X_bX_c$
$R_7 : C \to cC$	$C \to X_cC$
$R_8 : C \to c$	C → c (FNC)

6 - Quebrar cadeias longas (p/ cada produção $A \to w$, onde o lado direito w tem comprimento maior que 2 ($|w| > 2$)):

↳ essa produção é substituída por uma sequência de produções binárias (criadas agora)

↳ se $w = Bw'$ ⇒ a produção é substituída por: $A \to BT$, $T \to w'$ (T: um novo não-terminal)





Forma Final (Exemplo):

Inicial:

G: $S \to aABC \mid a$
$A \to aA \mid a$
$B \to bbB \mid bc$
$C \to cC \mid c$

passos 1, 2, 3 e 4 não são necessários p/ esta G.

passo 5:

$S \to X_a ABC \mid a$
$A \to X_a A \mid a$
$B \to X_b X_b B \mid X_b X_c$
$C \to X_c C \mid c$

novos não-terminais p/ os terminais:
$X_a \to a$
$X_b \to b$
$X_c \to c$

passo 6:

$S \to X_a T_1 T_2 \mid a$
$A \to X_a A \mid a$
$B \to X_b T_3 B \mid X_b X_c$
$C \to X_c C \mid c$
$X_a \to a$
$X_b \to b$
$X_c \to c$
$T_1 \to A T_2$
$T_2 \to BC$
$T_3 \to X_b B$

Final (FNC):

$S \to X_a T_1 T_2 \mid a$
$A \to X_a A \mid a$
$B \to X_b T_3 \mid X_b X_c$
$C \to X_c C \mid c$
$X_a \to a$
$X_b \to b$
$X_c \to c$
$T_1 \to A T_2$
$T_2 \to BC$
$T_3 \to X_b B$

O Lema do Bombeamento p/ LLC garante que um símbolo não-terminal (variável) se repete em algum caminho na árvore de derivação longa.

Lema do Bombeamento (pumping lema)

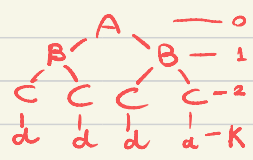
↳ ferramenta teórica usada p/ provar que uma determinada linguagem **não** pertence à classe das LLCs.

> Seja L uma linguagem livre de Contexto gerada por uma gramática G que está na FNC, e seja K o número de símbolos não-terminais de G:

↳ Se existir uma string $w \in L$ (∈ pertencente à) tal que seu comprimento $|w| > 2^{K-1}$, então w pode ser decomposto na forma $w = pqvty$, de modo que as seguintes condições sejam verdadeiras p/ p, q, v, t, y:

1) $w = pqvty$
2) $|qvt| \leq 2^{K-1}$ → (comprimento da seção bombeável)
3) $|q| + |t| > 0$ → (garante que q ou t contenham pelo menos 1 terminal)
4) $w_i = pq^i vt^i y \in L(G), \forall i \geq 0$

como a altura da árvore é maior que o nº total de não-terminais, deve haver pelo menos um símbolo não terminal que se repete no maior caminho raiz-folha.



$|w| = 2 \cdot 2^K = 2^{K-1}$ (valor máximo do comprimento de w, pq é uma árvore binária!)

Como a FNC Justifica a Aplicação do Lema:

A FNC é indispensável porque impõe uma restrição geométrica à Árvore de Derivação (AD) que permite a identificação de repetições:

1. Estrutura Binária: Como a gramática G está na FNC, toda árvore de parse (T) gerada por G é uma árvore binária.

2. Relação Altura/Comprimento: Em uma árvore binária gerada por uma gramática em FNC com altura n, o número de folhas (ou seja, o comprimento da string w) é, no máximo, 2^{n-1}, ou seja, $|w| \leq 2^{n-1}$.

3. Garantia de Repetição: Se escolhermos uma string z longa o suficiente, tal que seu comprimento |z| seja maior que 2^{K-1} (onde K é o número de não-terminais), sua árvore de parse deve ter altura $n \geq K+1$.

4. O Princípio da Casa dos Pombos: Como a altura da árvore é maior do que o número total de não-terminais (K), deve haver pelo menos um símbolo não-terminal que se repete no caminho de maior tamanho que parte da raiz e chega a alguma folha.

5. A Ciclo de Produção (A→vAx): Essa repetição implica que existe um não-terminal A que deriva recursivamente a si mesmo (A⇒vAx), permitindo que a parte intermediária (v e x) seja omitida (i=0) ou repetida i vezes (i≥0), gerando novos strings que permanecem em L

* Depois de definir uma palavra w que cumpra os requisitos,
↳ testamos todos os casos/possibilidades de divisão em 5 partes.
↳ o lema só é satisfeito quando pelo menos uma divisão gera uma decomposição válida → (e não que todas divisões funcionem)

> Aplicação do Lema - Prova por Contradição

Assumir a Hipótese ⇒ anunciamos que a linguagem L é LLC.

★ Isso implica que deve existir um n_i K tal que todas as strings w em L com $|w| > 2^{K-1}$ possam ser decompostas satisfazendo a LLC.

★ $L = \{a^i b^i c^i \mid i \geq 0\}$

↳ Suponha que L é uma LLC gerada por uma GLC em FNC com K símbolos não terminais.

↳ Tome $w = a^{2^{K-1}} b^{2^{K-1}} c^{2^{K-1}}$. Como $|w| = 3 \cdot 2^K$, $|w| > 2^{K-1}$, vindo assim, pelo lema do bombeamento:

1) $w = pqvty$
2) $|qvt| \leq 2^{K-1}$
3) $|q| + |t| > 0$
4) $w_i = pq^i vt^i y \in L(G), \forall i \geq 0$

1° CASO: $|a^{2^K}|b^{2^K} c^{2^K}$ — p | qvt | y

(q e t tem só 1 terminal)

⇓

bombeando a's, t a's mas = b's e c's, logo $w_i \notin L$

2° CASO: $|a^{2^K}|b^{2^K}|c^{2^K}$ — p | q | v (ab) | t | y

(q e t tem 2 terminais)

⇓

$a^{2^{K-1}-2} | a^i | ab | b^i | b^{2^{K-1}-2} c^{2^{K-1}}$ — p | q | v | t | y

= bombeando a's e b's, t a's e b's mas = c's.

= análogo p/ b's e c's

= impossível p/ a's e c's

$w_i \notin L$

3° CASO: $|a^{2^K} b^{2^K}| c^{2^K}$ — p | qvt | y

(q e t tem 2 terminais simultaneamente)

= bombeando perde-se a ordem, logo $w_i \notin L$

↳ abcabc...

(como fica isso?)

4° CASO: $|a^{2^K} b^{2^K} c^{2^K}|$ — p | qvt | y

(q e t contêm os 3 terminais)

= impossível! pois $|qvt| \leq 2^{K-1}$, e se q ou t contêm a's, b's e c's, q ou $t \geq 2^{K-1} + 2$

PROVA POR ABSURDO!

(por contradição) =

FNC:
- $S \to \lambda$
- $A \to -$
- $A \to BC$

Segundo Exemplo:

★ $L(G) = \{ a^n b^n \mid n \geq 0 \}$

G: $S \to aSb \mid \lambda$

1° ↓ criar S'

G_1: $S' \to S \mid \lambda$
$S \to aSb \mid \lambda$

2° ↓ eliminar λ (exceto o inicial)

G_2: $S' \to S \mid \lambda$
$S \to aSb \mid ab$

3° eliminar produções unitárias (ex: $S' \to S$) (regras de cadeia)

G_3: $S' \to aSb \mid ab \mid \lambda$
$S \to aSb \mid ab$

não temos inúteis nessa

4° ↓ isolar terminais

não podemos misturar terminal c/ não-term. ou ter + de 1 term!

G_4: $S' \to X_a S X_b \mid X_a X_b \mid \lambda$
$S \to X_a S X_b \mid X_a X_b$
$X_a \to a$
$X_b \to b$

5° ↓ quebrar cadeias longas (não-binárias)

Aplicação do L.B:

Se L é LLC, existe ⇐ um p tal que toda string $w \in L$ com $|w| > p$ pode ser dividida em 5 partes $w = pqvty$, satisfazendo:

1) $|qvt| \leq p$ → sendo que $p = 2^{K-1}$, sendo K o nº de variáveis de G.
2) $|q| + |t| > 0$ (ou ≥ 1)
3) $w_i = pq^i vt^i y \in L(G), \forall i \geq 0$

FNC

G_5: $S' \to X_a T_1 \mid X_a X_b \mid \lambda$
$S \to X_a T_1 \mid X_a X_b$

(não importa a ordem)
- $X_a \to a$
- $X_b \to b$
- $T_1 \to S X_b$

↳ escolhendo: $w = a^{2^K} b^{2^K}$

1° CASO: $\underset{p}{|} a^{2^K} \underset{qvt}{|} b^{2^K} \underset{y}{|}$ OU $a^{2^K} \underset{p}{|} b^{2^K} \underset{qvt}{|} \underset{y}{|}$

= bombeando, o nº de a's +, mas de b's =. $w_i \notin L(G)$.

= bombeando, o nº de b's +, mas de a's =. $w_i \notin L(G)$.

2° CASO: $\underset{p}{|} a^{2^K} b^{2^K} \underset{qvt}{|} \underset{y}{|}$ = bombeando a e b em q e t, removemos a ordenação inicial

abababab

3° CASO: $\underset{p}{a^1} \underset{q}{|} a^{\frac{2^K}{2}-2} \underset{v}{|} ab \underset{t}{|} b^{\frac{2^K}{2}-2} \underset{y}{|} b^1$ = bombeando ambos, q e t, w_i mantém sua integridade ao manter a proporção igual de a's e b's. $w_i \in L(G)$.

(proporcionar qvt no meio da string)

* dividir por 2 garante que as letras estão divididas igualmente.

$L(G)$ é LLC !!

Gabi :)

Tabela de Classificação de Linguagens Formais

I. Linguagens Regulares (LR) — Memória Finita (AF)

#	Linguagem (L)	Descrição Matemática	Classificação	Critério Chave	Confusão Comum na Prova
1	Sequências simples	$a^*b^*c^*$	LR	Reconhecida por AF. Não exige **pareamento** ou **contagem relativa** entre blocos [1].	Confundida com a^nb^n se a contagem não for exigida.
2	Divisibilidade	$L = \{w \in \{0,1\}^* \mid w \text{ é divisível por } 6\}$	LR	A divisibilidade pode ser verificada usando estados que registram o resto da divisão (memória finita) [2].	Condições aritméticas finitas são geralmente LR.

II. Linguagens Livres de Contexto (LLC) — Memória de Pilha (AP)

A. LLC Determinísticas (LAD)

#	Linguagem (L)	Descrição Matemática	Classificação	Critério Chave	Confusão Comum na Prova
3	Pareamento Simples	$\mathbf{L = \{0^n 1^n \mid n \geq 0\}}$	LLC (APD)	Exige **contagem** e **pareamento**; impossível para AF [1] [...]. É reconhecida por APD, pois a transição de empilhar `0`s para desempilhar `1`s é determinística (ocorre ao ler o primeiro `1`) [3] [4].	É o exemplo clássico de não-LR [1].
4	Contagem Comparativa	$L = \{0^m 1^n \mid m \leq n\}$	LLC (APD)	Requer que o número de `0`s seja menor ou igual ao de `1`s. Pode ser resolvida por um APD que empilha todos os `0`s e verifica se a pilha está vazia *antes* da fita de `1`s [8].	Provada **Não-LR** pelo Lema do Bombeamento para LRs [9] [10].
5	Contagem com Marcador	$L = \{0^m 1^n \# \mid m \geq n\}$	LLC (APD)	O marcador # torna a decisão **determinística**; o APD sabe exatamente quando parar de empilhar ou desempilhar para fazer a comparação [8] [11].	Se o marcador # fosse removido, ela se tornaria a L_6 (LLC Não-Determinística).

II. Linguagens Livres de Contexto (LLC) — Memória de Pilha (AP)

B. LLC Não-Determinísticas (LLC \ LAD)

#	Linguagem (L)	Descrição Matemática	Classificação	Critério Chave	Confusão Comum na Prova
6	Contagem Inversa (Palindromos)	$\mathbf{L} = \{\mathbf{w}\mathbf{w}^{\mathbf{R}} \mid \mathbf{w} = \mathbf{w}^{\mathbf{R}}\}$ (Palindromos)	**LLC (APN)**	Exige **comparação simétrica reversa**. O AP tem que **adivinhar** o centro w_{meio} (transição λ que testa se w é par/impar) [12].	É LLC, mas não LAD, pois o APD não pode se dar ao luxo de ter **transições compatíveis** para adivinhar o centro [3].
7	Contagem Livre de Ordem	$L = \{w \mid n_0(w) = n_1(w)\}$ (Balanceamento)	**LLC (APN)**	Exige **contagem igual** sem ordem fixa. O APN usa a pilha como um balanço (empilha 0s, desempilha 1s, ou vice-versa).	**Não é Regular (LR)** [13] [14]. Gramática GLC para esta linguagem é $P \to 0P1P \mid 1P0P \mid \lambda$ [15].
8	Contagem Inversa Simples	$L = \{0^m 1^n \mid m \geq n\}$	**LLC (APN)**	O AP precisa empilhar 0s e garantir que, no mínimo, todos os 1s sejam pareados. Reconhecida por APN, mas a falta de marcador torna difícil a construção de um APD [8] [16].	Provada **Não-LR** [17] [18].

$L = \{ww^R \mid w \in \{a,b\}^*\} = LLC$

$S \to aSa \mid bSb \mid \lambda$

abaabba

III. Fora de LLC (Não-LLC) —

#	Linguagem (L)	Descrição Matemática	Classificação	Critério Chave	Confusão Comum na Prova
9	Múltiplas Contagens Similares	$\mathbf{L = \{a^i b^i c^i \mid i \geq 0\}}$ lema do bombeamento	**Não-LLC**	Requer **pareamento de três variáveis** [19]. Uma única pilha só pode parear duas contagens de forma independente. Exige Autômato com **2 Pilhas** [20].	**Confusão CRÍTICA.** O Lema do Bombeamento para LLCs é usado para provar a falha [19].
10	Interseção de Contagens	$L = \{a^n b^n c^m\} \cap \{a^m b^n c^n\}$ Fechamento da operação do LLC	**Não-LLC**	A interseção resulta em $\{a^n b^n c^n\}$ [21], que é a linguagem Não-LLC L_8. LLCs **não são fechadas sob interseção** [21].	Confusão com o fechamento: embora LRs sejam fechadas sob interseção [22], LLCs não são [21].
11	Duplicação de Sub-strings	$\mathbf{L = \{ww \mid w \in \{a,b\}^*\}}$ (String duplicada) ou $ww^R w$ não exige só simetria, mas que a 2a metade repita a 1a com exatidão ⇓ comparação + complexa lema do bombeamento	**Não-LLC**	Exige **comparação posicional** (o primeiro símbolo de w deve ser igual ao primeiro símbolo do segundo w). A pilha não consegue acessar o fundo da memória para essa comparação [21].	Provada **Não-LR** [23] [24]. A falha no reconhecimento por AP se deve à necessidade de lembrar a primeira metade w *após* desempilhá-la (ou empilhamento duplo complexo).
12	Quadrados Perfeitos	$L = \{a^{n^2} \mid n \geq 0\}$	**Não-LLC**	Requer **contagem complexa** que exige muito mais que a memória empilhável do AP [25] [26].	Provada **Não-LR** [25] [26].
13	Linguagens Primas	$L = \{a^n \mid n \text{ é primo}\}$	**Não-LLC**	Exige **verificação de propriedades aritméticas** que excedem a capacidade de um AP [27].	Provada **Não-LR** [27].

Requer 2+ Pilhas (Múltiplos Contadores)

#	Linguagem (L)	Descrição Matemática	Classificação	Critério Chave	Confusão Comum na Prova
14	Contagem com Soma	$L = \{a^m b^n c^k \mid m = n+k\}$ duas pilhas $b^{(n)} c^{(k)}$ } a 1a lembra do a vamos um dos	**LLC**	O AP pode empilhar n de b e k de c (ou usar estados para gerenciar as duas contagens) e depois parear com m de a.	**Parece Não-LLC**, mas a soma pode ser gerenciada em uma única pilha. Provada **Não-LR** [28] [29].
15	Duas contagens diferentes	$L = \{a^i b^j c^k \mid i = j \text{ ou } i = k\}$ $\{a^i b^j c^k\} \cup \{a^i b^j c^i\}$ LLC LLC i vai pra pilha = LLC	**LLC**	É a união de duas LLCs: $\{a^i b^i c^*\}$ e $\{a^k b^* c^k\}$. LLCs são fechadas sob **união** [19].	**Não é Determinística (LAD).** O APN precisa adivinhar se $i = j$ ou $i = k$.

$G: S_1 \to aSbC \mid \lambda$ ∪ $G: S_2 \to aSc \mid aBc \mid \lambda$

$C \to Cc \mid \lambda$ $B \to Bb \mid \lambda$

$S' \to S_1 \mid S_2$ } cria um símbolo inicial novo

$S_1 \to aSbC \mid \lambda$

$S_2 \to aScB \mid aBc \mid \lambda$ } repete as regras

$C \to Cc \mid \lambda$

$B \to Bb \mid \lambda$

são simples, então não existe ambiguidade

(formato)

GR vs GLC

GLC → pode ser ambígua

GR: produções só à direita ou só à esquerda

$S \to \lambda$
$A \to aB$
$B \to a$

ou

$S \to \lambda$
$A \to Ba$
$B \to a$

mas:

$A \to B$ não precisa (descartável)

$A \to BBX$ (+ de 2 variável)

$A \to A$ X (recursão – inválida)

↳ loop (não produz terminal)

GLC: qualquer string de terminais e não-terminais

$S \to \lambda$
$S \to aSb$
$S \to SS$
$S \to Sa \mid aS$
$S \to a$

GLC em FNC:

$S \to \lambda$
$A \to AB$
$B \to b$

mas $A \to Aa$ X

2. Linguagens Reconhecidas por Autômatos de Pilha (AP)

Os Autômatos de Pilha (AP) – incluindo o determinístico (APD) e o não determinístico (APN) – reconhecem as Linguagens Livres de Contexto (LLC). As linguagens reconhecidas por um AP podem ser definidas por três critérios, dependendo se o critério de aceitação é o estado final, a pilha vazia, ou ambos.

Seja um AP M com estado inicial i e alfabeto de entrada Σ, a relação de movimento é representada por $\vdash^*$ (que significa zero ou mais passos de transição) [9]. A configuração instantânea de um AP é dada pela tripla $[e, w, p]$, onde e é o estado atual, w é o restante da palavra de entrada, e p é o conteúdo da pilha (topo → fundo) [9] [...].

a. Reconhecimento por Estado Final e Pilha Vazia (Padrão $L(M)$)

Esta definição exige que o AP termine em um estado final ($f \in F$) e com a pilha vazia (λ):

Para um Autômato de Pilha Determinístico (APD) $M = (E, \Sigma, \Gamma, \delta, i, F)$:

$$L(M) = \{w \in \Sigma^* | [i, w, \lambda] \vdash^* [f, \lambda, \lambda] \text{ para algum } f \in F\}$$

P: _|_|

$i \to F$ (zero ou + passos de transição) $\to f$

Esta mesma notação $L(M)$ é usada para definir a Linguagem Reconhecida por um Autômato de Pilha Não Determinístico (APN) 13 ... :

$$L(M) = \{w \in \Sigma^* | [i, w, \lambda] \overset{*}{\vdash} [f, \lambda, \lambda] \text{ para algum } f \in F\}$$

Uma palavra w é dita ser aceita por M se ela satisfizer essa condição 11 12 .

b. Reconhecimento por Estado Final Apenas ($L_F(M)$)

Esta definição requer que o AP termine em um estado final ($f \in F$) após consumir toda a entrada, independentemente do conteúdo da pilha ($y \in \Gamma^*$):

Para um Autômato de Pilha Não Determinístico (APN) M:

$$L_F(M) = \{w \in \Sigma^* | [i, w, \lambda] \overset{*}{\vdash} [f, \lambda, y] \text{ para algum } f \in F, y \in \Gamma^*\}$$

c. Reconhecimento por Pilha Vazia Apenas ($L_V(M)$)

Esta definição requer que o AP termine com a pilha vazia (λ) após consumir toda a entrada, independentemente do estado final ($e \in E$):

Para um Autômato de Pilha Não Determinístico (APN) $M = (E, \Sigma, \Gamma, \delta, i)$:

$$L_V(M) = \{w \in \Sigma^* | [i, w, \lambda] \overset{*}{\vdash} [e, \lambda, \lambda] \text{ para algum } e \in E\}$$

16 17

É importante notar que, para linguagens reconhecidas por APNs, o reconhecimento por pilha vazia e estado final ($L(M)$) é equivalente ao reconhecimento por estado final ($L_F(M)$), e L pode ser reconhecida por pilha vazia se $L \cup \{\lambda\}$ puder ser reconhecida por pilha vazia (L_V) 18 .